

Test Plan — Restful Booker API

Document Version: 1.0

Prepared By: QA Lead

Date: 2026-05-24

Project: Restful Booker API

API Documentation URL: <https://restful-booker.herokuapp.com/apidoc/index.html>

Objective

The objective of this test plan is to verify the Restful Booker API, a booking management system that supports the full lifecycle of hotel booking records — including creation, retrieval, update, partial update, deletion, and token-based authentication. The system is known to contain intentional defects; a primary objective is to systematically discover, document, and report all such defects in accordance with established defect management procedures. All test cases will be designed against the published API documentation at <https://restful-booker.herokuapp.com/apidoc/index.html> and executed manually using Postman. Automation coverage will be implemented in parallel using the REST Assured (Java) framework to support regression and CI/CD integration. Test execution spans the following API endpoints: `POST /auth`, `GET /booking`, `POST /booking`, `GET /booking/{id}`, `PUT /booking/{id}`, `PATCH /booking/{id}`, `DELETE /booking/{id}`, and `GET /ping`.

Scope

The following testing types constitute the scope of this test plan for the Restful Booker API. Each type addresses a distinct quality dimension of the system under test.

1. Functional Testing

- Verify the correctness and functionality of all seven API endpoints as per the API documentation.
- Test all CRUD operations — Create, Read, Update, Partial Update, and Delete — for booking records.
- Validate user authentication and token generation via the `POST /auth` endpoint.
- Confirm that protected endpoints (`PUT`, `PATCH`, `DELETE`) enforce authentication requirements.

2. Data Validation Testing

- Ensure the API correctly validates all input fields, rejecting malformed or missing data with appropriate error responses.
- Test boundary values for numeric fields (`totalprice`), string length fields

(firstname, lastname, additionalneeds), and date fields (checkin, checkout).

- Validate the accuracy, completeness, and format of data returned in all API responses.

3. Error Handling Testing

- Verify that appropriate HTTP status codes and descriptive error messages are returned for invalid, incomplete, or malformed requests.
- Check that error responses do not expose sensitive system information, stack traces, or internal configuration details.
- Validate the API's ability to handle unexpected input types, null values, and empty payloads without crashing or returning 500 errors.

4. Performance Testing

- Assess API response times for all endpoints under normal operating conditions and identify any endpoints that exceed acceptable thresholds.
- Measure throughput and evaluate the API's scalability when subjected to increasing request volumes.
- Identify performance bottlenecks in read-heavy and write-heavy operation patterns.

5. Security Testing

- Conduct assessments to identify common vulnerabilities including SQL injection, cross-site scripting (XSS), and improper input handling.
- Validate that all API communication is conducted over HTTPS and that no sensitive data is transmitted in plain text.
- Verify that authentication tokens are validated correctly on all protected endpoints and that unauthorized requests are rejected with HTTP 401 or 403 responses.
- Test for insecure direct object reference (IDOR) vulnerabilities by attempting to access or modify bookings using IDs belonging to other sessions.

6. Integration Testing

- Verify data consistency and workflow correctness across related endpoints — for example, confirming that a booking created via `POST /booking` is retrievable via `GET /booking/{id}` and appears in the `GET /booking` list.
- Test the end-to-end flow: authenticate → create booking → retrieve booking → update booking → delete booking → verify deletion.
- Validate that the `GET /ping` health check reflects the true operational state of the system.

7. Compatibility Testing

- Test the API across all supported client environments, including Windows 10

(Chrome, Firefox, Edge), macOS (Safari), Android (Chrome), and iOS (Safari).

- Verify that API responses conform to the JSON content-type standard and are correctly parsed by clients across platforms.
- Validate header handling and cookie-based authentication across different browser environments.

8. Documentation Review

- Assess the clarity, completeness, and accuracy of the API documentation at the published URL.
- Verify that every documented endpoint, parameter, request body, and response structure matches actual API behavior.
- Identify and log discrepancies between documented and observed behavior as documentation defects.

9. Load Testing

- Evaluate the API's behavior and stability under high concurrent user loads, simulating multiple users simultaneously performing CRUD operations.
- Identify the point at which API response time or error rate degrades to unacceptable levels.
- Assess the impact of load on data integrity — ensuring concurrent writes do not produce corrupted or duplicate booking records.

10. Regression Testing

- Execute a defined regression test suite after every bug fix or API update to confirm that previously passing test cases continue to pass.
- Automate regression coverage using REST Assured to enable rapid re-execution across builds.
- Maintain a regression baseline tied to the API documentation version.

11. Edge Case Testing

- Test extreme and boundary scenarios such as booking dates where `checkin` equals `checkout`, `totalprice` of zero, and `depositpaid` set to false.
- Validate API behavior when optional fields (e.g., `additionalneeds`) are omitted entirely from the request body.
- Test with unusually long string inputs, special characters, Unicode characters, and empty string values.

12. Concurrency Testing

- Assess the API's behavior when multiple clients simultaneously attempt to update or delete the same booking record.
- Verify that race conditions do not result in data corruption, double-deletions, or

inconsistent state.

- Test simultaneous token generation requests from multiple clients against `POST /auth`.

13.Ad Hoc Testing

- Perform unscripted exploratory testing sessions to uncover hidden defects not covered by formal test cases.
- Focus exploratory sessions on areas of the API where known bugs have been reported or where behavior appears inconsistent.
- Document findings from ad hoc sessions as informal defect reports for triage.

14.Usability Testing

- Evaluate the API's developer experience — assessing whether endpoint naming, request/response structure, and error messages align with REST conventions and developer expectations.
- Review whether the API documentation provides sufficient guidance for first-time users to integrate successfully without external assistance.

15.Continuous Integration and Deployment (CI/CD) Testing

- Validate that the REST Assured automation suite integrates with the CI/CD pipeline and executes reliably on each build trigger.
- Verify that test failures in the pipeline correctly block deployments and generate actionable failure reports.
- Ensure that environment-specific configuration (base URLs, credentials) is externalized and does not require code changes between environments.

16.Performance Monitoring

- Implement response time tracking across all API endpoints to establish a performance baseline.
- Monitor for response time regressions introduced by new builds or deployments.
- Track error rate trends over time to identify systemic instability in the hosted environment.

17.Backup and Recovery Testing

- Validate that the API's data reset mechanism (documented as resetting every 10 minutes) functions correctly and restores the pre-loaded dataset.
- Verify data integrity post-reset — confirming that the 10 pre-loaded booking records are accurately restored.
- Assess the impact of in-progress test operations during a scheduled data reset.

18.Internationalization Testing

- Test the API's handling of booking records that contain non-ASCII characters in

string fields (`firstname`, `lastname`, `additionalneeds`).

- Validate correct processing of date formats across locale-specific representations for `checkin` and `checkout` fields.
- Verify that response payloads correctly encode and return Unicode content without data loss or corruption.

19. Rate Limiting Testing

- Test the API's behavior when a high volume of requests is sent in rapid succession from a single client to identify whether rate limiting is enforced.
- Verify the HTTP status code and response body returned when rate limits are exceeded (expected: HTTP 429 Too Many Requests, if implemented).
- Assess whether rate limiting applies uniformly across authenticated and unauthenticated request types.

20. Third-Party Integration Testing

- Validate the API's compatibility with Postman as a third-party client — including collection runner execution, environment variable substitution, and pre-request script functionality.
- Verify that the REST Assured automation framework correctly integrates with the API and produces reliable, repeatable results.
- Assess the API's behavior when accessed from JIRA-integrated testing tools used for defect lifecycle management.

Inclusions

The following testing areas are explicitly included within the scope of this test plan, mapped to the Restful Booker API's CRUD operations and supporting capabilities.

1. Create (POST) Operations — `POST /booking`

- Test the creation of new bookings using fully valid request bodies containing all required fields: `firstname`, `lastname`, `totalprice`, `depositpaid`, `bookingdates.checkin`, `bookingdates.checkout`.
- Verify that the API returns HTTP 200 with the created booking object and a unique `bookingid` upon successful creation.
- Test creation requests with missing mandatory fields and verify HTTP 400 or equivalent error responses.
- Validate that newly created bookings are immediately retrievable via `GET /booking/{id}` and appear in the `GET /booking` list.

2. Read (GET) Operations — `GET /booking` and `GET /booking/{id}`

- Test retrieval of the full list of booking IDs via **GET /booking** without query parameters.
- Test filtered retrieval using supported query parameters: **firstname**, **lastname**, **checkin**, and **checkout**.
- Test retrieval of a specific booking by valid **bookingid** via **GET /booking/{id}** and verify all fields match the created booking.
- Test retrieval with a non-existent **bookingid** and verify the appropriate HTTP 404 response.
- Validate that the response body structure matches the documented booking object schema.

3. Update (PUT) Operations — **PUT /booking/{id}**

- Test full update of an existing booking using valid authentication (Cookie token or Basic Auth) and a complete replacement request body.
- Verify that the API returns HTTP 200 with the updated booking data upon successful full update.
- Test update requests submitted without authentication credentials and verify HTTP 403 or 401 responses.
- Validate that the booking record reflects the updated values when subsequently retrieved via **GET /booking/{id}**.
- Test updates to a non-existent booking ID and verify the appropriate error response.

4. Partial Update (PATCH) Operations — **PATCH /booking/{id}**

- Test partial updates of existing bookings by submitting request bodies containing only a subset of booking fields.
- Verify that fields not included in the PATCH request body remain unchanged in the stored booking record.
- Test PATCH requests without authentication and verify HTTP 403 or 401 responses.
- Validate that partial updates to date fields correctly enforce logical constraints (e.g., **checkin** must precede **checkout**).

5. Delete (DELETE) Operations — **DELETE /booking/{id}**

- Test deletion of an existing booking using valid authentication credentials.
- Verify that the API returns HTTP 201 upon successful deletion (as documented).
- Validate that the deleted booking is no longer retrievable via **GET /booking/{id}** (expected: HTTP 404).
- Test deletion requests without authentication and verify HTTP 403 or 401 responses.
- Test deletion of a non-existent booking ID and verify the appropriate error response.

6. Authentication — **POST /auth**

- Test token generation using valid credentials (`username: admin, password: password`) and verify that a non-empty token string is returned.
- Test token generation with invalid credentials and verify an appropriate error response or error message in the response body.
- Validate that the generated token successfully authenticates requests to protected endpoints (`PUT`, `PATCH`, `DELETE`) when supplied as a `Cookie` header.
- Validate Basic Authentication as an alternative authentication method for protected endpoints.

7. Health Check — **GET /ping**

- Verify that the health check endpoint returns HTTP 201 when the service is operational.
- Include ping verification as part of the smoke testing phase to confirm environment readiness before full test execution.

8. Boundary Testing

- Test the `totalprice` field with values at the minimum (0), maximum integer boundary, and negative values.
- Test `firstname` and `lastname` fields with single-character values, maximum permissible length strings, and empty strings.
- Test `bookingdates` with identical `checkin` and `checkout` values, and with `checkin` set later than `checkout`.

9. Concurrency Testing

- Simulate concurrent `PUT` and `DELETE` requests targeting the same `bookingid` to assess race condition handling.
- Execute parallel `POST /booking` requests and verify that each request generates a unique `bookingid`.
- Test simultaneous `GET /booking` list requests under concurrent write load to verify response consistency.

10. Data Validation

- Test all string fields with special characters, SQL injection strings, script tags, and excessively long inputs.
- Test numeric fields (`totalprice`) with string values, floating-point values, and null to verify type enforcement.
- Test boolean field (`depositpaid`) with string representations (`"true"`, `"false"`) and integer values (1, 0).
- Verify that date fields (`checkin`, `checkout`) reject non-date strings and

incorrectly formatted date inputs.

11.Authentication and Authorization

- Test all protected endpoints with an expired or invalid token and verify HTTP 403 responses.
- Test protected endpoints with no authentication header or cookie and verify rejection.
- Verify that unauthenticated users can access read-only endpoints (GET /booking, GET /booking/{id}) without a token.
- Test token reuse across multiple requests to confirm token validity is not single-use.

12.Error Handling

- Test all endpoints with malformed JSON request bodies and verify HTTP 400 responses.
- Test requests with unsupported HTTP methods (e.g., DELETE /booking without an ID) and verify HTTP 404 or 405 responses.
- Verify that error response bodies are structured consistently and do not expose internal system details.

13.Security Testing

- Test `firstname`, `lastname`, and `additionalneeds` fields with SQL injection payloads and verify the API does not execute or reflect them.
- Test for XSS by inserting script tags into string fields and verifying responses do not reflect executable scripts.
- Verify HTTPS enforcement — confirm that HTTP requests are redirected to HTTPS or rejected.
- Test for IDOR by attempting to access or delete bookings created in other sessions using guessed IDs.

14.Performance Testing

- Measure baseline response times for all endpoints under single-user conditions.
- Execute bulk read requests against GET /booking and measure response time degradation at scale.
- Evaluate POST /booking throughput by measuring the number of successful creations per unit time.

15.Integration Testing

- Validate the complete end-to-end booking flow: authenticate → create → read → update → partially update → delete → verify deletion.
- Verify that GET /booking filter parameters (`firstname`, `lastname`, `checkin`, `checkout`) correctly return only matching records following a

POST /booking creation.

16.Regression Testing

- Execute the full automated REST Assured test suite after each reported bug fix to confirm resolution without regression.
- Maintain a minimum regression suite covering the critical path: authentication, CRUD operations, and error handling.

17.Documentation Review

- Compare each documented request/response structure against actual API behavior for all seven endpoints.
- Log discrepancies between documented HTTP status codes and observed responses as documentation defects in JIRA.
- Verify that the API documentation URL remains accessible and current throughout the test cycle.

18.Load Testing

- Execute concurrent user simulations targeting GET /booking, POST /booking, and PUT /booking/{id} to assess stability under load.
- Identify the concurrency threshold above which the API's error rate or response time exceeds acceptable limits.

19.Compatibility Testing

- Execute Postman collections against the API from Windows 10 (Chrome, Firefox, Edge), macOS (Safari), Android (Chrome), and iOS Safari environments.
- Verify consistent API behavior and response parsing across all supported platforms.

20.Usability Testing

- Assess whether the API adheres to RESTful conventions (resource naming, HTTP method semantics, status code usage).
- Evaluate the completeness and accuracy of the Swagger documentation as a self-service integration guide.

21.CI/CD Testing

- Integrate the REST Assured test suite into the CI/CD pipeline and verify execution on each build trigger.
- Confirm that test failures in CI/CD block promotion to subsequent environments.

22.Rate Limiting Testing

- Send high-frequency requests to POST /auth and GET /booking to test for rate-limit enforcement.
- Verify appropriate HTTP 429 responses if rate limiting is implemented.

23.Backup and Recovery Testing

- Validate the API's 10-minute automatic data reset cycle by observing state before and after reset.
 - Verify that all 10 pre-loaded booking records are present and correct following each reset.
-

Test Environments

The test environments for the Restful Booker API are defined as follows. All testing will be conducted against HTTPS endpoints using JSON content type. The QA environment serves as the primary execution environment for functional, exploratory, and automation testing. The Pre-Production environment will be used for final validation prior to any deployment activities.

Operating Systems and Browsers:

- Windows 10 — Google Chrome (latest), Mozilla Firefox (latest), Microsoft Edge (latest)
- macOS — Safari (latest)
- Android Mobile OS — Google Chrome (latest)
- iOS — Safari (latest)

Network: Wi-Fi and wired Ethernet connections at standard broadband speeds. Mobile testing conducted over LTE/4G connectivity.

Authentication: Token-based authentication via `POST /auth` endpoint using credentials `admin / password`. Basic Authentication header also supported as an alternative.

Name	Env URL
QA	https://restful-booker.herokuapp.com/apidoc/index.html
Pre Prod	https://restful-booker.herokuapp.com/apidoc/index.html

Note: The Restful Booker API resets its data state every 10 minutes, restoring the default set of 10 pre-loaded booking records. Test execution sequences must account for this reset cycle to ensure test data created during a session remains available for the duration of dependent test steps.

Defect Reporting Procedure

A defect is identified when any observed API behavior deviates from the specification documented at <https://restful-booker.herokuapp.com/apidoc/index.html>, violates a standard REST convention, exposes a security vulnerability, or produces an error inconsistent with the system's documented error handling. All defects, including those attributable to known

intentional bugs in the Restful Booker system, are subject to the reporting procedure below.

Identification Criteria: A defect report is raised when (1) an API endpoint returns an HTTP status code that differs from the documented response code, (2) a response body field is missing, null, or incorrectly typed relative to the documented schema, (3) a protected endpoint accepts a request without valid authentication, (4) an unprotected endpoint rejects a valid unauthenticated request, (5) the system returns an HTTP 500 error for any input, or (6) response times consistently exceed 3,000 milliseconds under normal load conditions.

Reporting Steps: Upon identifying a defect, the reporting tester must (1) capture the complete request — including HTTP method, URL, headers, and request body; (2) capture the complete response — including status code, headers, and response body; (3) record the expected behavior as specified in the API documentation; (4) document exact reproduction steps in numbered sequence; (5) attach relevant screenshots or Postman response exports; (6) log the defect in JIRA using the designated bug template, assigning the correct component tag (Frontend, Backend, or DevOps); and (7) notify the relevant POC as specified in the table below.

Triage and Prioritization: All defects are triaged by the QA Lead within one business day of submission. Severity levels are assigned as follows — Critical: system crash or data loss; High: core functionality broken; Medium: incorrect behavior with a workaround available; Low: cosmetic or documentation issue. Priority is assigned jointly by the QA Lead and the relevant development POC based on business impact and sprint capacity.

Tools: JIRA is the primary defect tracking and lifecycle management tool. Postman is used for capturing request/response evidence. Screenshots are captured using the system Snipping Tool. Defect status is communicated in daily end-of-day status emails to development management.

Communication: The QA Lead sends a daily defect summary report to development management at the close of each business day, detailing defects opened, in progress, and resolved during that day.

Defect Process	POC
Frontend	[Frontend Dev Name]
Backend	[Backend Dev Name]
DevOps	[DevOps Name]

Test Strategy

Step 1: Test Case Design

Test cases for the Restful Booker API are designed using the following techniques, applied as appropriate to each endpoint and testing area:

Equivalence Class Partitioning (ECP): Input fields are partitioned into valid and invalid equivalence classes. For example, `totalprice` is partitioned into valid positive integers, zero, negative integers, and non-numeric types. One representative value from each partition is tested to maximize coverage efficiency.

Boundary Value Analysis (BVA): Test cases target the exact boundaries of acceptable input ranges — for numeric fields (minimum, maximum, minimum-1, maximum+1), for string fields (empty string, single character, maximum allowed length, maximum+1), and for date fields (minimum valid date, maximum valid date, same-day checkin/checkout).

Decision Table Testing: Decision tables are constructed for scenarios involving multiple input conditions — specifically for authentication-dependent endpoints, where the combination of authentication state (present/absent/invalid) and booking existence (exists/does not exist) produces distinct expected outcomes.

State Transition Testing: The booking lifecycle is modeled as a state machine — states include: Not Created → Created → Updated → Partially Updated → Deleted. Transitions are tested in sequence, out of sequence, and with invalid transitions (e.g., updating a deleted booking).

Use Case Testing: End-to-end use cases are defined to simulate realistic client interactions, including: (1) a guest creates a booking and then modifies it; (2) an administrator authenticates and deletes a specific booking; (3) a client searches for bookings by guest name and retrieves matching records.

Error Guessing: Drawing on QA expertise and knowledge of common REST API defect patterns, test cases are created for scenarios likely to expose bugs — including empty body POST requests, mismatched content-type headers, duplicate booking IDs, and token reuse after data reset.

Exploratory Testing: Unscripted exploratory sessions are conducted to probe areas of the API where formal test case coverage is insufficient, leveraging tester judgment and domain knowledge to uncover unexpected behavior.

Step 2: Testing Procedure

Upon receipt of a build or environment confirmation, the following testing procedure is executed:

Smoke Testing: The QA team begins with smoke testing to confirm that the core functionalities of the Restful Booker API are operational. Smoke tests cover: GET /ping returning HTTP 201, POST /auth returning a valid token, GET /booking returning a list of booking IDs, and POST /booking successfully creating a booking. If smoke testing fails, the build is rejected and returned to the development team. Full testing does not commence until a stable build passes all smoke tests.

In-Depth Test Execution: Once a stable build is confirmed, the full test case suite is executed across all endpoints and testing types defined in the Scope and Inclusions sections. Multiple testers execute test cases simultaneously across the supported environments (Windows, macOS, Android, iOS). Defects identified during execution are logged in JIRA immediately, categorized by severity and priority, and assigned to the relevant POC.

Defect Retesting and Regression: Upon notification of a bug fix, the originally failing test cases are retested (retesting). A regression suite is then executed to confirm that the fix has not introduced new defects in previously passing functionality. REST Assured automation is used to accelerate regression execution.

Test Cycle Repetition: Test cycles — smoke, in-depth execution, defect retesting, and regression — are repeated until the API achieves the quality standard defined in the Exit Criteria. The QA

Lead sends an end-of-day defect status email to development management at the close of each execution day.

Testing types executed within each cycle include: Smoke Testing, Sanity Testing, Functional Testing, Regression Testing, Retesting, Security Testing, Performance Testing, and Exploratory Testing.

Step 3: Best Practices

The following best practices govern all testing activities for this project:

Context-Driven Testing: All testing decisions — including test case prioritization, depth of coverage, and defect severity assignment — are made in the context of the Restful Booker API's specific characteristics: a publicly hosted, intentionally buggy booking management API used for QA training. Testing effort is concentrated where defect probability is highest, based on the documented intent of the system.

Shift Left Testing: Testing activities commence at the earliest possible stage. Test case design begins in parallel with API documentation review, and smoke tests are executed as soon as an environment is available, without waiting for full feature completion. This approach reduces the cost of defect discovery by identifying issues earlier in the delivery cycle.

Exploratory Testing: Alongside scripted test execution, dedicated exploratory testing sessions are scheduled throughout each sprint. Exploratory sessions are time-boxed and focused on high-risk areas — authentication handling, concurrent modification, and error response consistency — where the documented intent to include bugs makes structured exploration particularly valuable.

End-to-End Flow Testing: Complete user journeys are tested as integrated scenarios, not just as isolated endpoint calls. The primary end-to-end flow — authenticate, create booking, retrieve booking, update booking, delete booking, verify deletion — is executed as a single sequence to validate that all API components work correctly in combination and that data flows correctly between endpoints.

Test Schedule

The following schedule governs the testing activities for the Restful Booker API across two sprints. Specific dates are to be confirmed with the project stakeholders and inserted by the team prior to sign-off.

Task	Dates
Creating Test Plan	[INSERT DATE RANGE]
Test Case Creation	[INSERT DATE RANGE]
Test Case Execution	[INSERT DATE RANGE]
Summary Reports Submission	[INSERT DATE]

Sprint Duration: The testing effort is planned across **2 sprints** to ensure adequate coverage of all CRUD operations, security, performance, and regression testing activities.

Test Deliverables

The following documents and artifacts will be produced and delivered as outputs of the testing engagement:

- **Test Plan Document** — This document, reviewed and signed off by the client prior to test execution commencement.
 - **Test Scenarios Document** — A high-level list of testing scenarios covering all endpoints and testing types, reviewed and signed off by the client.
 - **Test Cases Document** — Detailed test cases with steps, input data, expected results, and actual results, executed in Postman and documented in the designated tracking tool.
 - **REST Assured Automation Suite** — Java-based automated test scripts implementing the regression test suite, integrated with the CI/CD pipeline.
 - **Postman Collection** — Exported Postman collection containing all manually executed API requests, organized by endpoint and testing type.
 - **Defect Reports** — JIRA defect tickets for all identified bugs, including severity, priority, reproduction steps, and evidence (screenshots, request/response logs).
 - **Daily Defect Status Reports** — End-of-day email summaries to development management detailing defects opened, in progress, and resolved.
 - **Test Summary Report** — Final report summarizing test execution results, defect metrics, coverage achieved, and quality assessment, submitted at test closure.
-

Entry and Exit Criteria

The following entry and exit criteria govern progression through each phase of the Software Testing Life Cycle for the Restful Booker API project.

Requirement Analysis

Entry Criteria

- The testing team has received the API documentation URL (<https://restful-booker.herokuapp.com/apidoc/index.html>) and any supplementary specification documents.
- Access to the QA environment has been confirmed and the `GET /ping` endpoint returns HTTP 201.
- The project scope, objectives, and sprint schedule have been communicated to the QA team.

Exit Criteria

- All API endpoints listed in the documentation have been reviewed and understood by the

QA team.

- All ambiguities, discrepancies, or missing specifications have been raised with the relevant stakeholders and resolved or logged as open queries.
 - The Test Plan has been drafted and submitted for client review.
-

Test Execution

Entry Criteria

- The Test Plan, Test Scenarios, and Test Cases documents have been reviewed and signed off by the client.
- The QA environment is stable and accessible — confirmed by successful smoke test execution (GET /ping, POST /auth, GET /booking).
- JIRA project and bug tracking templates are configured and accessible to all team members.
- Postman collections and REST Assured project structure are set up and version-controlled.

Exit Criteria

- All planned test cases have been executed at least once.
 - All defects discovered during execution have been logged in JIRA with complete reproduction steps and evidence.
 - Test Case Execution Reports are complete and shared with stakeholders.
 - All Critical and High severity defects have been retested following developer fixes.
 - Regression testing has been executed and results documented.
-

Test Closure

Entry Criteria

- Test Case Execution Reports and Defect Reports are finalized and available for review.
- All Critical and High severity defects have been resolved, retested, and closed, or explicitly deferred with stakeholder approval.
- All test deliverables (Postman collection, REST Assured suite, daily reports) have been produced.

Exit Criteria

- The Test Summary Report has been completed, reviewed, and submitted to the client.
- The client has acknowledged and accepted the test summary findings.
- All open JIRA defects have been triaged and assigned a resolution status (fixed, deferred, or accepted risk).

- Lessons learned have been documented and shared with the team for future sprint improvements.
-

Tools

The following tools are used throughout the testing engagement for the Restful Booker API:

- **Postman** — Primary tool for manual API test case execution, exploratory testing, and collection management. Used to send requests to all endpoints and capture request/response evidence.
 - **REST Assured (Java)** — Automation framework used to implement the regression test suite and integrate with the CI/CD pipeline.
 - **JIRA** — Bug tracking and defect lifecycle management tool. Used to log, triage, assign, track, and close all defects identified during testing.
 - **Mind Map Tool** — Used during test case design to visually map API endpoints, test scenarios, and coverage areas.
 - **Snipping Tool** — Used to capture screenshots of API responses and defect evidence for inclusion in JIRA defect reports.
 - **Microsoft Word / Excel** — Used for test plan documentation, test case tabulation, and summary reporting where project management tools do not cover the required format.
 - **Git / Version Control** — Used to manage and version the REST Assured automation codebase and Postman collection exports.
-

Risks and Mitigations

The following risks have been identified for the Restful Booker API testing engagement, along with defined mitigation strategies.

Risk	Mitigation
Non-Availability of a Resource: A QA team member becomes unavailable during test execution due to illness, leave, or attrition, reducing testing capacity.	Maintain a backup resource plan by cross-training at least one additional team member on the test case suite and JIRA workflows. Identify a backup tester prior to the test execution phase and ensure all test artifacts are documented with sufficient detail to enable handover.
Environment Instability — Build URL Not Accessible: The Restful Booker API's hosted environment (https://restful-booker.herokuapp.com) is unavailable due to hosting issues, maintenance, or the Heroku platform's free-tier limitations, preventing test execution.	When the environment is unavailable, resources are redeployed to test case creation, documentation review, or REST Assured automation development — activities that do not require live environment access. The QA Lead monitors environment availability via <code>GET /ping</code> before each daily session and communicates status to the team.

Risk	Mitigation
Insufficient Time for Testing: Sprint scope or defect volume exceeds the planned test schedule, leaving insufficient time to complete all testing types within the allocated sprints.	Prioritize test execution based on risk — Critical and High severity functional and security test cases are executed first. Communicate scope risk to stakeholders at the first indication of timeline pressure and negotiate scope adjustments or resource augmentation. Leverage REST Assured automation to accelerate regression testing and free manual testers for exploratory and edge case coverage.
Data Reset Interference: The Restful Booker API's automatic 10-minute data reset deletes test data created during a session, causing dependent test steps to fail mid-execution.	Design test cases to be self-contained wherever possible, creating required test data at the start of each test sequence. Schedule extended test flows to execute within a single reset window. Document the reset cycle in the test environment notes and include a warning step in the test execution procedure.
Known Intentional Bugs Causing False Failures: The Restful Booker API is documented as intentionally containing defects, which may cause confusion between intentional bugs and newly introduced regressions.	Maintain a defect baseline log distinguishing confirmed intentional defects from newly discovered regressions. Tag JIRA issues appropriately (e.g., known-bug vs. regression) and ensure that the regression suite explicitly excludes test cases that exercise known open defects until those defects are resolved.

Approvals

The testing engagement for the Restful Booker API will not progress from one phase to the next without formal client approval of the corresponding deliverable. The following document types require explicit written sign-off from the designated client stakeholder before the associated testing activities may proceed. Approval is to be provided via email confirmation or signature on the document, as agreed with the client at project initiation.

The team will submit the **Test Plan** for client approval before test case design begins. Client sign-off on the Test Plan confirms agreement on scope, environments, schedule, defect reporting procedure, and entry/exit criteria. Following Test Plan approval, the team will submit the **Test Scenarios** document for client review, confirming that the planned testing coverage aligns with client expectations. Once Test Scenarios are approved, the **Test Cases** document will be submitted for sign-off, confirming that the detailed test steps, input data, and expected results meet the client's quality standards before execution commences. Upon completion of test execution, the team will submit the **Test Summary Report** for client acknowledgement, confirming acceptance of the test results, open defect disposition, and overall quality assessment. Testing activities will only advance to the next phase once the corresponding approval has been received and documented.